



Overview

1. Benefit of JIMI Open API, distributor or application vendor could provide tracking service to customers by calling the API and use your own GUI client, App or Website, this might improve the localization or user experience.
2. Your App or web client should connect to your application server, rather than connect to JIMI API Server directly , use your application server connect to JIMI Server instead.

Usage

1. Contact us to apply your appKey and appSecret, you need to provide your account.
2. According to this document to implement your application server to obtain access_token.
3. Calling other interfaces with access_token to fulfil your business logic.

Conventions

Encoding, format & Timezone

UTF-8 and JSON is used by default.

Context-Type: default application/json charset=utf-8

The API use UTC (GMT +0)time in default: format yyyy-MM-dd HH:mm:ss

Common response fields

Key	Type	Required	Description
code	int	Yes	resul code

Key	Type	Required	Description
message	string	No	Description

Result code description

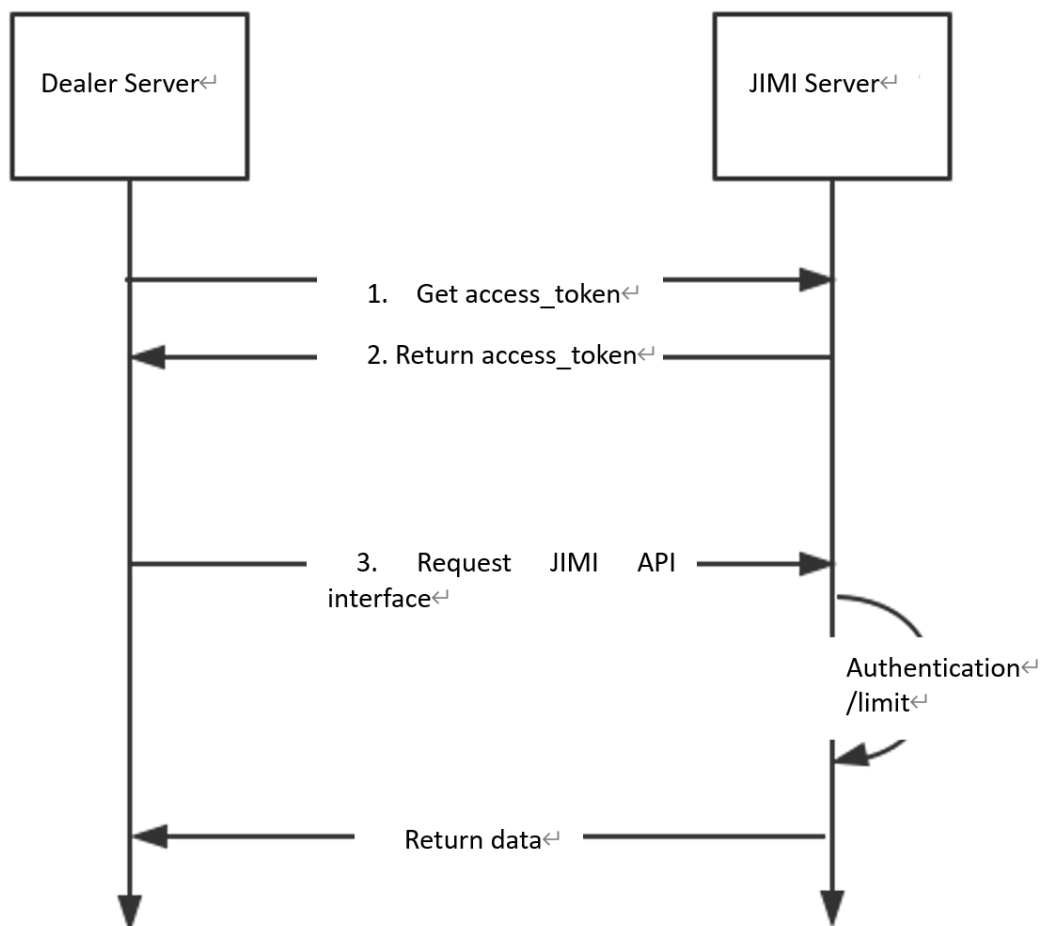
Code	Value	Description
-1	-1	The system is busy
0	0	success
1XXX	1001	Parameter error (lack of required parameters or format error). See interface description for details
	1002	Illegal user/illegal device (not their own or subordinate account or device)
	1003	Repeat operation
	1004	Illegal access, token exception! (Token failure or nonexistent)
	1005	Illegal access, IP access exceeds limit!
	1006	Illegal access, too frequently request!
	1007	Illegal access, request method error!
	1008	Illegal access, abnormal incoming!
	12001	platform account creation failed
	12002	device transfer failed
	12003	Geo-fence creation failed
	12004	Geo-fence deletion failed
	12005	Fail to send command

Error code :

Code	Value	Description
Create account	213	Account already exist
	214	Account does not exist

Code	Value	Description
	215	User type error. Possible reasons: 1.User type is empty; 2. User type does not exist; 3. Current login user do not have permission to create new type of user.
Device transfer	217	Target user does not exist
	218	User can only transfer/sale devices to sub-account
	219	IMEI list is illegal
	220	Number of IMEI exceeds limitation.
Geo-fence	41001	Exceed max number of Geo-fences supported
	41002	Fence name is already exists
	41003	The device is not online
	41004	Geo-fence operation failed
Command	225	time out
	226	Parameter error
	227	The command is not executed correctly
	228	The device is not online
	229	Network error, connection error, etc.
	238	Device interrupted
	240	Data format error
	243	Not supported by device
	252	The device is busy
	30010	INCORRECT INSTRUCTION_PASSWORD

API Work Flow



API Features

Interface type	Method	Description
Access Control	jimi.oauth.token.get	Get access token
	jimi.oauth.token.refresh	Refresh access token
Device Management	jimi.user.device.list	List all devices of sub-account
	jimi.track.device.detail	Get device detail information
	jimi.user.device.expiration.update	Update user expiration date
	jimi.open.device.update	Update vehicle information by IMEI
	jimi.open.device.move	Move devices
	jimi.open.device.bind	Bind app user

Interface type	Method	Description
	jimi.open.device.unbind	Unbind app user
	jimi.device.group.create	Create device group
	jimi.device.group.update	Edit device group
	jimi.device.group.delete	Delete device group
	jimi.device.group.list	Get device group list of an account
Tracking Function	jimi.user.device.location.list	Get location of devices by account
	jimi.device.location.get	Get the location of device(s)
	jimi.device.location.URL.share	Get sharing location URL
	jimi.device.track.mileage	Get the mileage data of devices
	jimi.device.track.list	Get the track data of device
	jimi.lbs.address.get	Wi-Fi, Base Station locating analysis
	jimi.open.platform.report.parking	Get parking/idling data of devices
	jimi.device.location.getTagMsg	Get the location of TAG device
Media Function	jimi.device.live.page.url	Get Device Live Steaming Page URL
	jimi.device.meida.cmd.send	Send media instruction
	jimi.device.history.cmd.send	Send history video instruction
	jimi.device.jimi.media.URL	Get Device JIMI Photo or Video URL
	jimi.device.media.event.URL	Get Device Media Event URL
Command Management	jimi.open.instruction.list	Get command list supported by device

Interface type	Method	Description
	jimi.open.instruction.send	Send command to device
	jimi.open.instruction.result	Get results of command execution
	jimi.open.instruction.raw.send	Send raw command data to device
Geofencing Function	jimi.open.device.fence.create	Create Geo-fence for IMEI
	jimi.open.device.fence.delete	Delete Geo-fence for device
	jimi.open.platform.fence.create	Create platform Geo-fence
	jimi.open.platform.fence.create	Edit platform Geo-fence
	jimi.open.platform.fence.delete	Delete platform Geo-fence
	jimi.open.platform.fence.bind	Geo-fence related device
	jimi.open.platform.fence.list	List platform geofences of an account
	jimi.open.platform.fence.detail	Query single fence information
Push Function	jimi.push.device.alarm	Receive Notification
	jimi.open.instruction.raw.receive	Push Received Raw Data
OBD	jimi.device.obd.list	Get the OBD data of devices
	jimi.device.obd.fault	Get the OBD fault data of devices
Reports	jimi.open.platform.fence.duration	Get entry and exit fence data of devices
	jimi.open.platform.report.trips	Get the trips report data of devices
	jimi.device.alarm.list	Get device alarm list

Interface type	Method	Description
	jimi.open.device.rfid.list	Get RFID reporting information
Account Management	jimi.user.child.list	List all sub-account
	jimi.user.child.create	Create sub-account
	jimi.user.child.del	Remove sub-account
	jimi.user.child.move	Move account
	jimi.user.child.update	Edit user information

Request URL

The unique request URL is:

Please use the URL of the corresponding node according to the node to which your OPEN API account belongs

Note:

TS: <http://open.10000track.com/route/rest>

TSP HK/SG: <https://hk-open.tracksolidpro.com/route/rest>

TSP EU: <https://eu-open.tracksolidpro.com/route/rest>

TSP US: <https://us-open.tracksolidpro.com/route/rest>

⚠ Different interfaces are identified by request parameter method.

Security

1. Client should get access_token first before calling the interface, which is generated by JIMI Server by calling jimi.oauth.token.get interface(method=jimi.oauth.token.get) with provided appKey and appSecret.
2. appKey and access_token are required to sign request parameters when calling API. JIMI server will validate the the request parameters by checking the sign value.

Common parameters

The parameters of each request should include common parameters and interface private parameters.

For example:

if you call the "jimi.oauth.token.get" interface, you need to provide : 7(common parameters) + 3(private parameters) = 10 (parameters, key/value)

Following are the common parameters:

Name	Type	Required	Description	Remark	Default
method	string	Yes	API interface name		
timestamp	string	Yes	Timestamp, format:yyyy - MM - dd HH:mm:ss. Plus or minus 10 minutes is allowed. e.g:2012-03-25 20:00:00	GMT(UTC) time	
app_key	string	Yes	appKey	from JIMI	
sign	string	Yes	A signature base on parameters, appKey, appSecret.		
sign_method	string	Yes	Optional, signature method. Available value: md5	md5	md5
v	string	Yes	Optional, specify the API version. System default 1.0, support version: 0.9, 1.0	0.9: no signature checking 1.0: check signature	
format	string	Yes	Optional, specify response format.		json

Note:

The parameter V uses a difference of 0.9 and 1.0:

Using 0.9 will not perform Sign signature verification

Using 1.0 will verify the Sign signature

Function description of Sign signature verification:

The data receiving end gets the transmission text, but needs to confirm whether the text is the content sent by the sender, and whether it has been tampered with in the middle.

Therefore, the receiver uses its own public key to decrypt the signature and obtains the digest of the text, then uses the same method as the sender to calculate the digest value of the text, and compares it with the decrypted digest, and finds that the two are exactly the same. It means that the text has not been tampered with.

Signature

To protect API calling from hacked, any API calling needs to be with a signature. JIMI server will check signature based on request parameters. Illegal signature request will be rejected. Signature algorithms supported is: md5 (sign_method is a common parameter mentioned above).

Following is the algorithm of signature:

1. Sort all request parameters with parameter key in alphabetical order (including common parameters and method specific parameters, but NOT include sign and byte type parameter. For example:

```
foo=1, bar=2, foo_bar=3, foobar=4
```

```
Result: bar=2, foo=1, foo_bar=3, foobar=4
```

2. Remove all equal sign and comma:

```
bar2foo1foo_bar3foobar4
```

3. Then concatenate appSecret to the before and end of the result string and get the md5 value. E.g:

```
md5(appSecret+bar2foo1foo_bar3foobar4+appSecret), the real string should like:
```

```
md5(h9Iri085eachcz4sn7gwnkh6j0jt0yz4bar2foo1foo_bar3foobar4h9Iri085eachcz4sn7gwnkh6j0jt0yz4)
```

⚠ Note that the string should be in UTF-8 encoding.

⚠ Note that the sign should be upper case string.

4. If the parameter value is a byte stream, it should be converted to hexadecimal. For example:

```
hex("helloworld".getBytes("utf-8")) = "68656C6C6F776F726C64"
```

MD5 is the 128-bit summary algorithm and is in hexadecimal. a hexadecimal character can represent four bits, so the signature string length is 32 hexadecimal characters.

```

public static String signTopRequest(Map<String, String> params, String seccode,
// 1: sort parameter key
String[] keys = params.keySet().toArray(new String[0]);
Arrays.sort(keys);
// 2:: Put all parameter names and parameter values together
StringBuilder query = new StringBuilder();
if (Constants.SIGN_METHOD_MD5.equals(signMethod)) {
    query.append(seccode);
}
for (String key : keys) {
    String value = params.get(key);
    if (StringUtils.isEmpty(key, value)) {
        query.append(key).append(value);
    }
}

// 3: use MD5/HMAC to encrypt
byte[] bytes;
if (Constants.SIGN_METHOD_HMAC.equals(signMethod)) {
    bytes = encryptHMAC(query.toString(), seccode);
} else {
    query.append(seccode);
    bytes = encryptMD5(query.toString());
}

// 4: convert binary to uppercase hexadecimal
return byte2hex(bytes);
}

public static byte[] encryptHMAC(String data, String seccode) throws IOException
    byte[] bytes = null;
    try {
        seccodeKey seccodeKey = new seccodeKeySpec(seccode.getBytes(Constants.CH
        Mac mac = Mac.getInstance(seccodeKey.getAlgorithm());
        mac.init(seccodeKey);
        bytes = mac.doFinal(data.getBytes(Constants.CHARSET_UTF8));
    } catch (GeneralSecurityException gse) {
        throw new IOException(gse.toString());
    }
    return bytes;
}

public static byte[] encryptMD5(String data) throws IOException {

```

```
        return encryptMD5(data.getBytes(Constants.CHARSET_UTF8));
    }

    public static String byte2hex(byte[] bytes) {
        StringBuilder sign = new StringBuilder();
        for (int i = 0; i < bytes.length; i++) {
            String hex = Integer.toHexString(bytes[i] & 0xFF);
            if (hex.length() == 1) {
                sign.append("0");
            }
            sign.append(hex.toUpperCase());
        }
        return sign.toString();
    }
}
```